# Limitations of Algorithmic Power

**(also covered in Levitin chapter 11)**

# Running Times: polynomial vs super-polynomial

| | 10 | 20 | 30 | 40 | 50 | 60 |
|---|---|---|---|---|---|---|
| $n$ | .00001 second | .00002 second | .00003 second | .00004 second | .00005 second | .00006 second |
| $n^2$ | .0001 second | .0004 second | .0009 second | .0016 second | .0025 second | .0036 second |
| $n^3$ | .001 second | .008 second | .027 second | .064 second | .125 second | .216 second |
| $n^5$ | .1 second | 3.2 second | 24.3 second | 1.7 minutes | 5.2 minutes | 13.0 minutes |
| $2^n$ | .001 second | 1.0 second | 17.9 minutes | 12.7 days | 35.7 years | 36 600 years |
| $3^n$ | .059 second | 58 minutes | 6.5 years | 385 500 years | $2\times10^{10}$ years | $1.3\times10^{15}$ years |

# What we'd like to know

**If we have** a polynomial time algorithm to solve a problem
is it the best possible algorithm for that problem?

**If we don't** have a polynomial time algorithm:
is it unsolvable ?
is it intractable ?
or are we just not clever enough ?

# Lower Bound Arguments

Ascertain the efficiency of an algorithm:

Establish its asymptotic efficiency class (for the worst case)

Lower bounds: try to establish **a limit** on efficiency of **any algorithm** solving a problem

# Lower Bound Example

**Incorrect:**

Selection sort is more effective than Towers of Hanoi, because it runs in $O(n^2)$ time, while Towers of Hanoi runs in superpolynomial time

Correct:

Selection sort is not efficient, as there exist sorting algorithms that run in $O(n \log n)$ time

# Lower Bound

**Best possible efficiency** any algorithm may have.

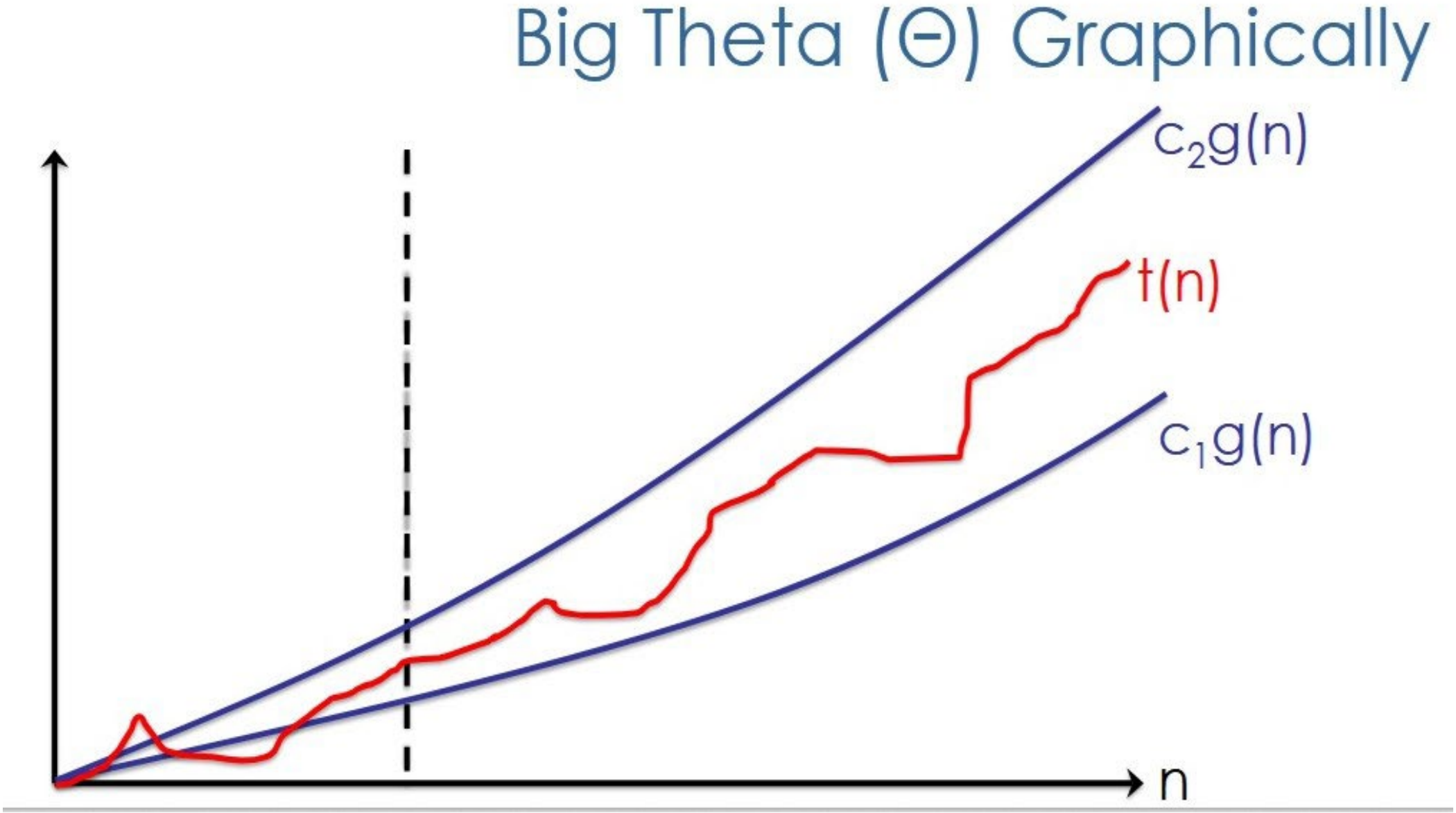
Serves as a reference for possible improvements:

- If the bound is **tight**, we already know an algorithm in same efficiency class as best possible algorithm — we can gain only constant factor improvement
- If there is a gap, then we can hope to get better algorithm or better lower bound

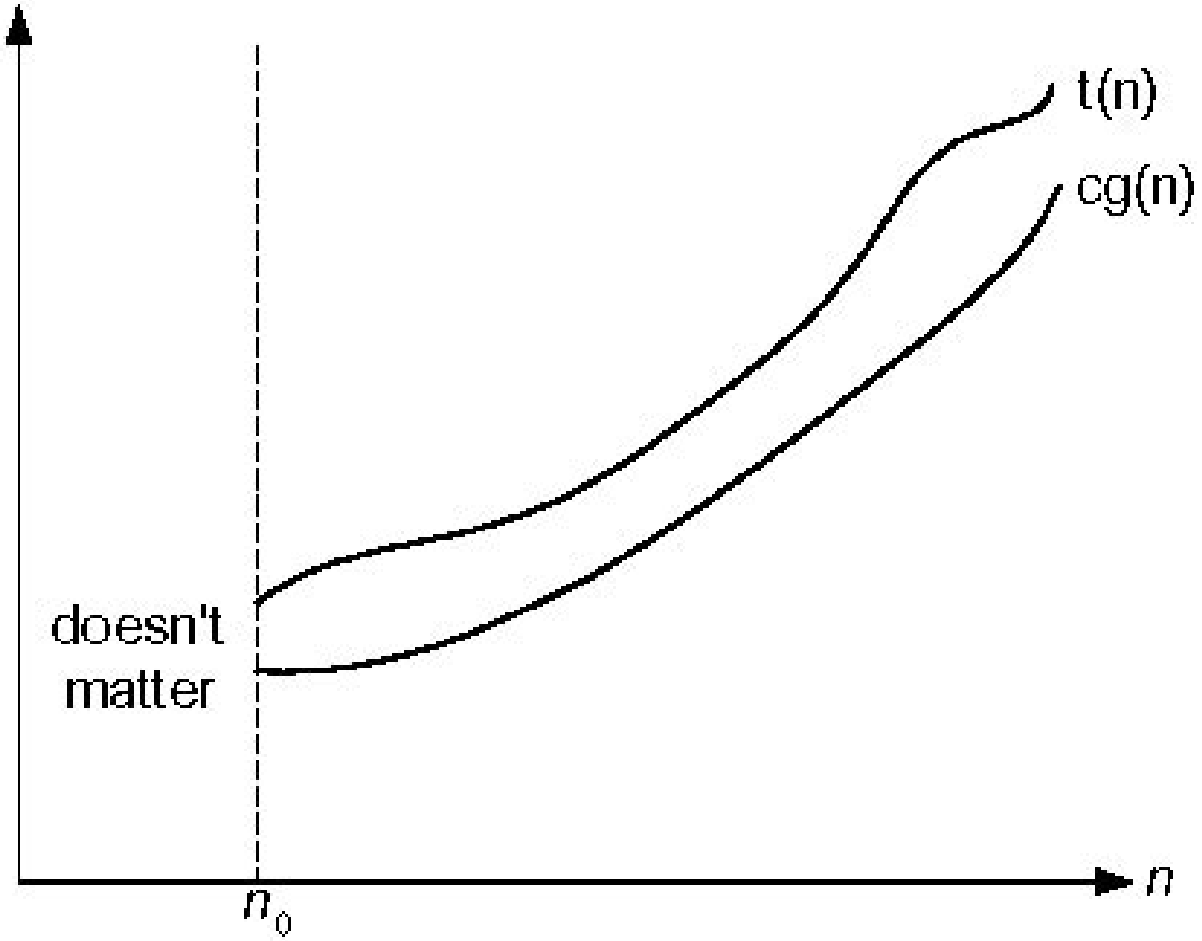
Lower bound can be

- an exact count
- an efficiency class Ω

# Efficiency notation - reminder



# Reminder - Big-Omega: $t(n) \in \Omega(g(n))$




Function t(n) is in $\Omega(g(n))$ if $t(n) \geq cg(n)$ for all $n \geq n_0$, where c is a positive constant; i.e. t(n) is bounded below by some positive constant multiple of g(n) for large n

# Upper and Lower Bounds

Example Problem: Searching on ordered list.

Sequential search algorithm is O(n)

This places an upper bound on the problem: meaning the complexity of the problem is no worse than O(n) because we already have an algorithm that solves it that fast

But now the binary search algorithm is O (log n)

This gives us a lower (better, more efficient) upper bound.

Can we do better ? Or would we waste time looking for a more efficient algorithm) ?
i.e. What is the inherent time complexity of the problem?

Would like to prove that problem P cannot be solved by any algorithm in less than, say, O (log n) time in the worst case. A proof like that would yield a lower bound for the problem.

For a problem P:

Upper bound U(n) is the minimum, over all known algorithms, of the worst-case complexity.

Lower bound L (n) is the minimum, over all possible algorithms, of the worst-case complexity.

Upper bound → # operations sufficient for solving P

Lower bound → # operations necessary for solving P (both in the worst case)

A problem is tight if the order of its lower and upper bounds is the same.

Problems for which upper and lower bounds do not meet (are not the same big-Oh) give rise to algorithmic gaps.

## Tight Problems and Algorithms Gaps.

A problem is tight if the order of its lower and upper bounds is the same.
Tight problems include searching ordered/unordered lists, and sorting.

E.g. Sorting (by comparisons)

Lower bound: $\lceil \log_2 n! \rceil$ comparisons

This is O (n log n).

Heapsort & Mergesort are both O (n log n)

Hence, both are of optimal order.

However, proving lower bounds is difficult.

Problems for which upper and lower bounds do not meet (are not the same big-Oh) given rise to algorithmic gaps.

# Establishing lower bounds: Trivial Lower Bound

Just count how much **input data** an algorithm reads and how much **output** it produces

- Algorithm must at least read in this data and write out the result
- This makes the trivial lower bound

# Finding Trivial Lower Bound: Examples

To generate all permutations of n items must be $\Omega(n!)$

- output size is n!

Multiplication of two n-by-n matrices is $\Omega(n^2)$

- size of input is $2n^2$, output is $n^2$

Finding the largest element in an array is $\Omega(n)$

- size of input is n

# Trivial Lower Bound Arguments

Trivial lower bounds are often **too low** to be useful:

- Travelling salesman's lower bound is $\Omega(n^2)$, because input is $n(n-1)/2$ distances
  - but no known algorithm for this with polynomial running time

Also, algorithm may not process all the input

- Lower bound for searching a sorted array would be $\Omega(n)$ because of the input size

In general, obtaining a non-trivial lower bound for even a simple problem is a very difficult task



# Adversary Arguments

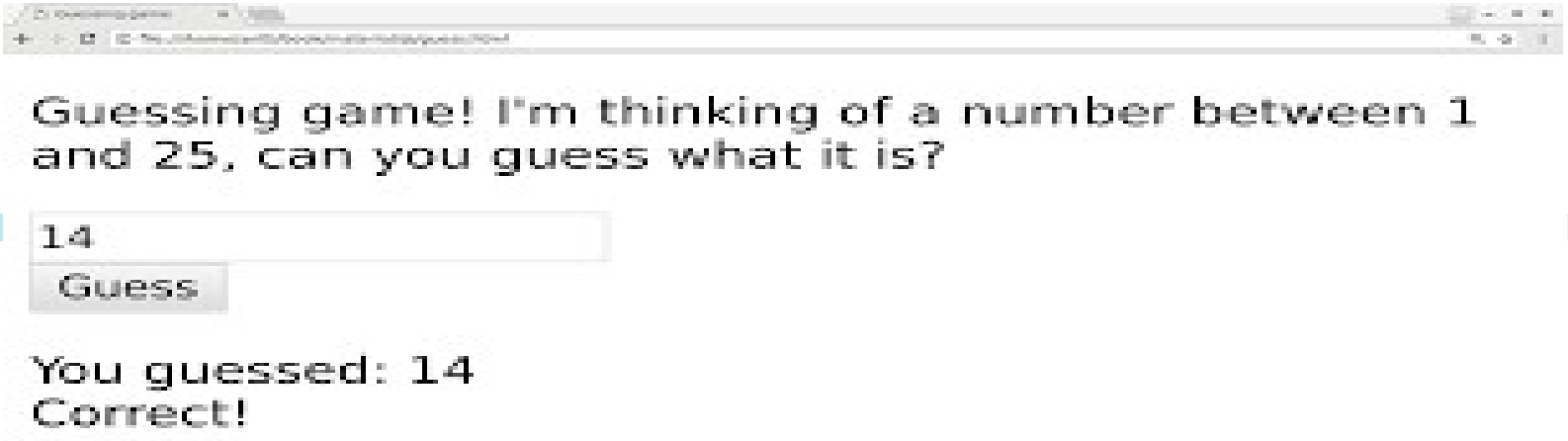
**Adversary method**: malevolent, but honest, adversary

malevolent: push down most time consuming path

honest: don't break any rules

Creates worst possible input for the specific algorithm to prove lower bound

# Adversary Arguments




The ***adversary*** forces any algorithm to use the ***max*** number of guesses. So e.g. if the first guess is 8, since there are more numbers between 8 & 25 than there are numbers between 1 and 8, the adversary says their number is larger than 8. And continues in this way, **but remains consistent** (i.e. simply adjusts the range of possible numbers he could have chosen to the max range still possible) Example for above 1-25 game:

**8? No:bigger! 20? No:smaller ! 16? No:smaller! 11? No:bigger!**

Currently adversary knows answer must be 12, 13, 14 or 15. Etc



# Adversary Arguments

**Adversary method**: malevolent, but honest, adversary

malevolent: push down most time consuming path

honest: don't break any rules

Creates worst possible input for the specific algorithm to prove lower bound

Example:

Merging 2 sorted lists: *when the algorithm compares* $a_i$ *and* $b_j$

**reply true for** $a_i < b_j$ **iff** $i < j$

$\Rightarrow b_1 < a_1 < b_2 < a_2 < \ldots < b_n < a_n$ *is the solution*

algorithm must compare 2n-1 adjacent pairs of elements

($b_1$:$a_1$, $a_1$:$b_2$, $b_2$:$a_2$, $a_2$:$b_3$ ... $b_n$:$a_n$).

# Adversary Arguments

Example: Merging two sorted lists of size $n$

adversary replies true for $a_i < b_j$ iff $i < j$

$\Rightarrow b_1 < a_1 < b_2 < a_2 < \ldots < b_n < a_n$ *is the valid output*

algorithm has to compare 2n-1 adjacent pairs of elements ($b_1$:$a_1$, $a_1$:$b_2$, $b_2$:$a_2$, $a_2$:$b_3$ ... $b_n$:$a_n$).

If one of these comparisons is *not performed,* e.g. ***$b_i$ and $a_i$ for some i***

then the following orders are consistent with all of the adversary answers:

$a_1 < b_1 < a_2 < \ldots < a_{i-1} < b_{i-1} <$ **$a_i < b_i < \ldots < b_n < a_n$**

$a_1 < b_1 < a_2 < \ldots < a_{i-1} < b_{i-1} <$ **$b_i < a_i < \ldots < b_n < a_n$**

cannot know which of these two orders is correct

cannot stop yet

all the 2n–1 comparisons are needed.

# Adversary Arguments

*the adversary makes absolutely **no** assumptions about the algorithm*

- any algorithm that does not do all the comparisons will fail.

## Prove, with an adversary argument, that any comparison-based algorithm for finding the largest among *n* given numbers must make *n-1* comparisons in the worst case

Adversary employs the following rule: **reply true to comparison $a_k > a_j$ *iff* $k > j$**

***then the only correct output will be element n.***

To produce this, the adversary would have to compare the maximum (element *n) to all other n-1 elements, so n-1 comparisons.*

If one of these has not been made, e.g., *element m has not been compared to n, then element m is a potential answer: keep guessing!*

Therefore, *n-1 is indeed a lower bound.*